

Informe técnico — Viajes Aventura

Integrantes: Lucas Espinoza Rojas

Asignatura: Programación Orientada a Objeto Seguro

Sistema: Viajes Aventura (MVP web)

Tecnologías: Python 3, Django 5, SQLite, HTML/CSS, ORM de Django

1. Resumen ejecutivo y problemática

Viajes Aventura reemplaza la gestión manual y desconectada de destinos, paquetes turísticos y reservas. El problema inicial producía información duplicada, riesgo de sobreventa, ofertas vencidas, exposición de datos personales y pérdida del precio acordado cuando cambiaba el catálogo.

La solución es una aplicación web Django con cuatro módulos: accounts, destinations, packages y reservations. Permite que el personal administre el catálogo y las reservas, mientras que los clientes se registran, consultan ofertas vigentes y reservan de forma privada. El sistema conserva los valores históricos al publicar un paquete y al emitir una reserva, y evita que las confirmaciones de pago excedan la capacidad.

Alcance del MVP: gestión de destinos, paquetes, reservas y estados de pago manuales. Quedan fuera de alcance el cobro automático, facturación electrónica, integraciones externas, API REST, aplicación móvil, mensajería y reportes operacionales.

2. Requerimientos identificados y priorizados

2.1 Requerimientos funcionales

ID	Requerimiento verificable	Prioridad	Evidencia de implementación
RF-01	El personal crea, consulta, edita y retira destinos; los usados se desactivan para preservar historia.	Must	destinations/models.py, destinations/views.py
RF-02	Un paquete borrador contiene entre 2 y 5 destinos disponibles y distintos.	Must	packages/services.py::validate_composition
RF-03	El personal publica paquetes con fechas, cupo y margen válidos.	Must	Package.clean, publish_package
RF-04	La publicación fija precio y contenido del paquete, aun si luego cambia un destino.	Must	PackageDestination y publish_package
RF-05	El cliente se registra con correo, contraseña, RUT y teléfono; inicia sesión con correo.	Must	accounts/models.py, accounts/forms.py

ID	Requerimiento verificable	Prioridad	Evidencia de implementación
RF-06	El catálogo muestra solamente paquetes publicados, habilitados y con salida futura.	Must	packages/views.py::catalog
RF-07	Un cliente autenticado crea una reserva pendiente y consulta exclusivamente sus propias reservas.	Must	reservations/views.py, create_reservation
RF-08	El personal cambia una reserva pendiente a pagada o rechazada; solo las pagadas consumen cupo.	Must	transition_reservation
RF-09	El sistema impide confirmar pagos que superen la capacidad y conserva el total cobrado.	Must	reservations/services.py
RF-10	El evaluador inicia el sistema mediante un único script y recibe guía de uso.	Should	iniciar_aplicacion.py, README.md
RF-11	La interfaz presenta navegación según rol, formularios con validación y estados vacíos comprensibles.	Should	templates/base.html y plantillas por módulo

2.2 Requerimientos no funcionales

ID	Requerimiento	Criterio de aceptación / evidencia
RNF-01 Seguridad	Autenticación basada en el sistema oficial de Django, contraseñas hashadas, sesiones, CSRF y autorización por rol.	AbstractUser, middleware CSRF, login_required, staff_member_required.

ID	Requerimiento	Criterio de aceptación / evidencia
RNF-02 Privacidad	No se deben mostrar RUT, teléfono ni contraseña en mensajes de error o listados.	Sanitización en accounts/views.py; pruebas de privacidad.
RNF-03 Integridad	Los datos críticos deben validarse en formulario, modelo, restricciones de BD y/o servicios transaccionales.	CheckConstraint, UniqueConstraint, full_clean() y transaction.atomic.
RNF-04 Confiabilidad	Una doble solicitud no debe crear dos reservas y la concurrencia no debe sobrepasar el cupo.	Token UUID único, transacción, reintentos acotados y prueba concurrente.
RNF-05 Usabilidad	La solución debe ser operable desde pantallas móviles y escritorio, con acciones y errores claros.	Especificación friendly-ui-design y plantillas responsivas.
RNF-06 Mantenibilidad	El código se separa por responsabilidad en apps, modelos, formularios, vistas, servicios y pruebas.	Estructura del repositorio y 16 pruebas automatizadas.
RNF-07 Portabilidad	Debe funcionar localmente con Python 3.10+ y SQLite.	requirements.txt, README.md, script de inicio.

2.3 Criterio de priorización

Se aplicó **MoSCoW**. Se clasificó como *Must* todo requisito que protege el negocio, el cupo, el precio histórico, la identidad o la privacidad. Los elementos de experiencia y entrega se clasificaron como *Should*, porque mejoran la evaluación y uso sin cambiar las reglas centrales. No se incluyeron requisitos *Could* o *Won't* dentro del MVP; las integraciones y automatizaciones se declararon explícitamente fuera de alcance para evitar ampliar el proyecto sin control.

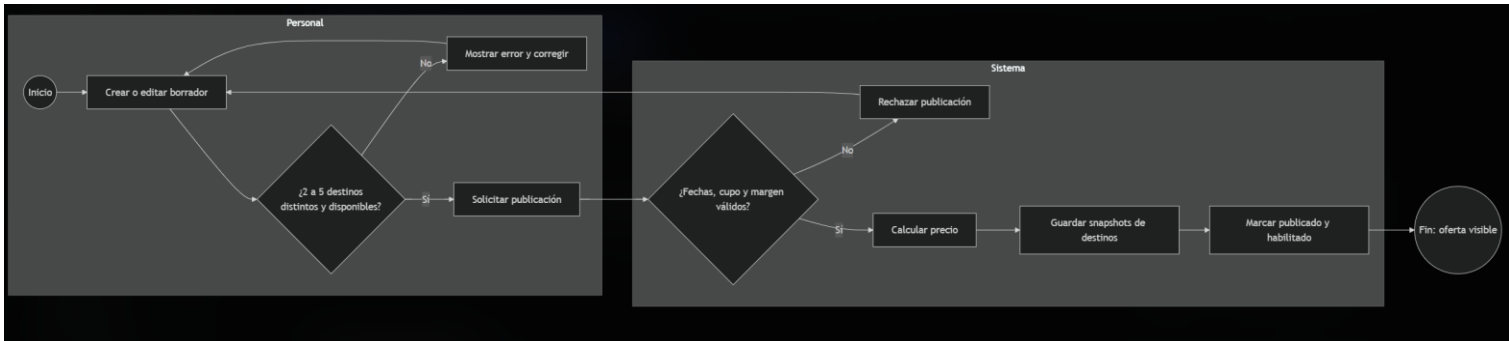
2.4 Alineación con el problema del cliente

Problema detectado	Respuesta del sistema	Reglas cubiertas
Catálogo manual con datos inválidos o duplicados	Destinos únicos, costos/duración positivos y validación del servidor.	R1, R2
Ofertas ambiguas o modificables luego de venderse	Paquetes con composición, fechas y precio fijados al publicar.	R3–R8
Clientes sin acceso seguro ni privacidad	Cuenta por correo, RUT validado, sesión y control de rol.	R9–R11, R17
Cobros y reservas sin historia confiable	Precio y total se copian al momento de reservar.	R12, R13
Riesgo de sobreventa	Solo pagos confirmados consumen cupo; transición transaccional y prueba concurrente.	R14–R16

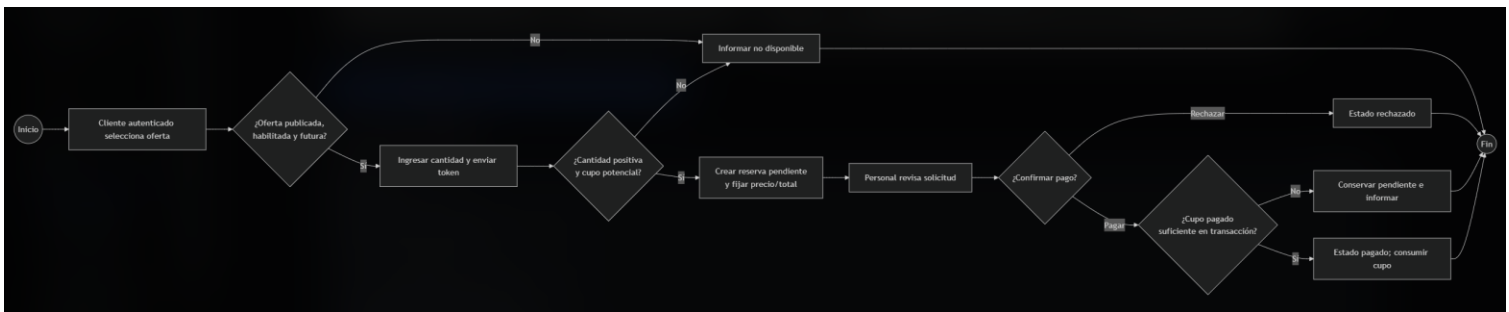
3. Modelamiento de procesos y UML

Los diagramas se incluyen como código editable: el bloque plantuml puede renderizarse con PlantUML y los bloques mermaid con un visor compatible. Esto permite revisar y corregir el modelo sin redibujarlo.

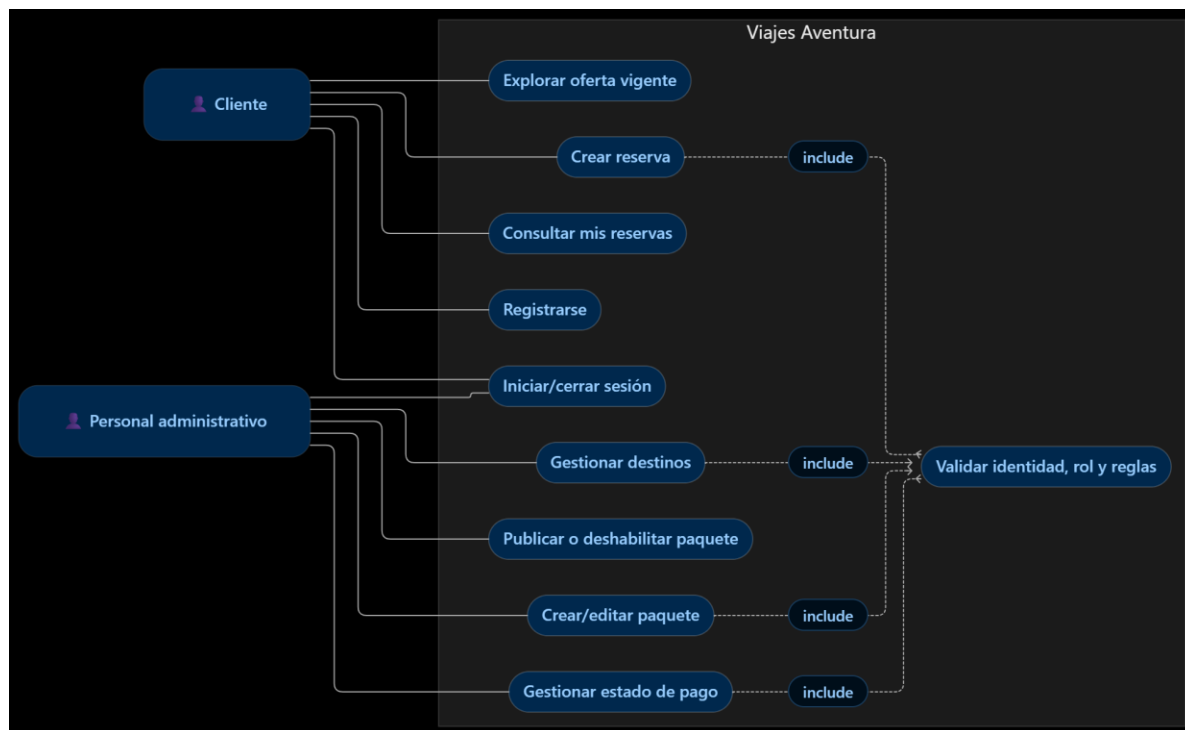
3.1 Proceso BPMN: publicación de un paquete



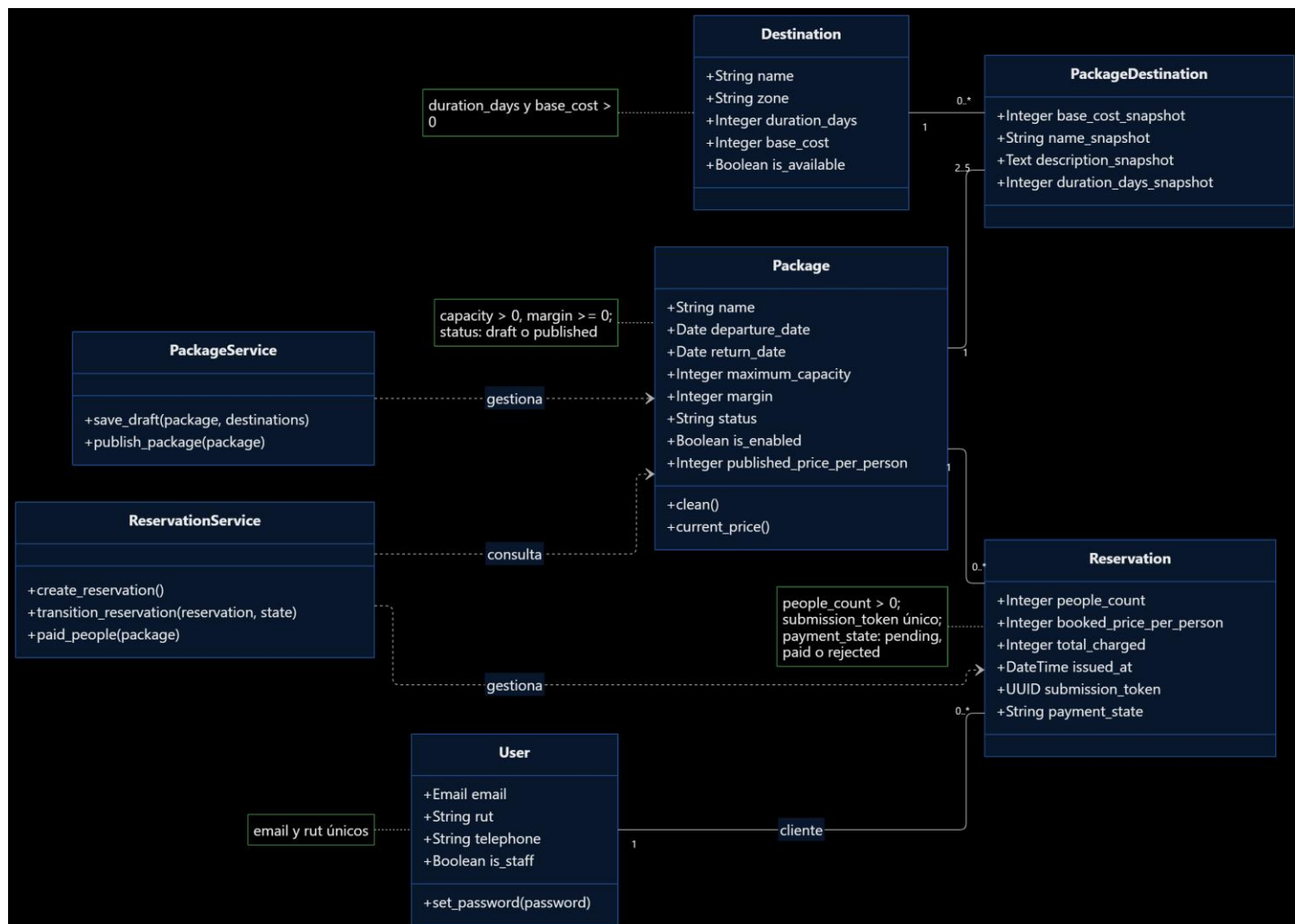
3.2 Proceso BPMN: reserva y confirmación de pago



3.3 Casos de uso UML



3.4 Diagrama de clases UML



3.5 Trazabilidad entre requisitos, modelos y pruebas

Regla	Caso/modelo asociado	Prueba automatizada
R1–R2	Destination y DestinationForm	DestinationTests.test_staff_crud_and_positive_values
R3–R7	Package, PackageDestination, publish_package	PackageTests.test_composition_price_and_immutable_snapshots
R8	destination_remove	DestinationTests.test_unused_deleted_and_used_deactivated
R9–R10	User, validate_rut, RegistrationForm	AccountTests.test_rut_validation_and_safe_registration
R11	Decoradores de acceso y consultas del dueño	AccountTests.test_private_routes_and_staff_boundary
R12–R13	Reservation y snapshots monetarios	ReservationTests.test_pending_is_idempotent_historical_and_private
R14–R16	transition_reservation, paid_people	ReservationTests.test_paid_capacity_and_rejection
R14 (concurrentia)	Transacción y reintentos SQLite	ConcurrentCapacityTests.test_simultaneous_confirmations_never_exceed_capacity

Regla	Caso/modelo asociado	Prueba automatizada
R17	Sanitización de formulario y vistas	AccountTests.test_rut_validation_and_safe_registration

La matriz completa también está disponible en tests/TRACEABILITY.md

4. Gestión ágil del proyecto

4.1 Roles y responsabilidades

Rol Scrum	Responsabilidad aplicada al proyecto
Product Owner	Priorizar reglas R1–R17, validar que la solución responda a Viajes Aventura y aceptar los entregables.
Scrum Master	Gestionar el tablero, remover bloqueos (migraciones, dependencias, conflictos de alcance) y facilitar retrospectiva.
Equipo de desarrollo	Modelar, implementar, probar, revisar seguridad y mantener la documentación técnica.
Evaluador/cliente	Ejecutar el flujo de inicio, probar casos principales y retroalimentar la experiencia.

4.2 Product Backlog priorizado

Orden	Historia de usuario / ítem	Prioridad	Criterio de terminado
1	Como cliente, quiero registrarme e iniciar sesión de forma segura.	Must	Correo/RUT únicos, contraseña hasheada, pruebas aprobadas.
2	Como personal, quiero administrar destinos válidos.	Must	CRUD protegido y retiro dependiente de uso.

Orden	Historia de usuario / ítem	Prioridad	Criterio de terminado
3	Como personal, quiero componer y publicar paquetes inmutables.	Must	2–5 destinos, precio/snapshots y validación.
4	Como cliente, quiero reservar ofertas disponibles sin exponer mis datos.	Must	Reserva privada, token único y total histórico.
5	Como personal, quiero confirmar pagos sin sobreventa.	Must	Transacción, cupo y prueba concurrente.
6	Como usuario, quiero una interfaz clara y adaptable.	Should	Navegación por rol, formularios y estado vacío.
7	Como evaluador, quiero iniciar el proyecto fácilmente.	Should	Script único, migraciones y README.

4.3 Sprint Backlog, tiempos y entregables

Sprint	Objetivo y tareas comprometidas	Entregable	Duración planificada
1	Configuración Django, modelo User, autenticación, RUT y permisos.	Cuentas operativas y pruebas de acceso.	1 semana
2	Modelo/CRUD de destinos, paquetes borrador y publicación con snapshots.	Catálogo administrativo y reglas R1–R8.	1 semana
3	Reservas, estados de pago, capacidad, privacidad y concurrencia.	Flujo cliente–personal y reglas R9–R17.	1 semana
4	Diseño visual, regresión, trazabilidad, inicio local e informe.	Suite de pruebas, README y documentación.	1 semana

Los cuatro sprints se consideran cerrados porque las tareas de los cambios archivados están marcadas como completadas y la suite final fue ejecutada.

5. Implementación técnica

5.1 Programación orientada a objetos y diseño

- **Encapsulamiento:** cada entidad define sus datos y reglas; por ejemplo, `Package.clean()` valida fechas y `Package.save()` bloquea modificaciones materiales tras publicar.
- **Abstracción y responsabilidad única:** las entidades (`User`, `Destination`, `Package`, `Reservation`) representan el dominio; los servicios concentran procesos transaccionales y las vistas atienden HTTP/autorización.
- **Herencia:** `User` extiende `AbstractUser` de Django y `UserManager` extiende `BaseUserManager`, reutilizando el framework de autenticación oficial.
- **Polimorfismo/extensibilidad:** `TextChoices` expresa estados de paquete y pago con una interfaz consistente; el ORM opera las entidades mediante la misma abstracción `models.Model`.
- **Relaciones:** `PackageDestination` modela la asociación muchos-a-muchos y conserva snapshots; `Reservation` relaciona cliente y paquete mediante claves foráneas protegidas.

5.2 Persistencia y CRUD

La persistencia se realiza mediante el ORM de Django sobre SQLite (`db.sqlite3`). Las migraciones crean tablas, claves foráneas, unicidad y restricciones de negocio. Los destinos se crean, listan, editan y retiran en `destinations/views.py`. Los paquetes se crean/editan como borrador y se publican mediante servicio; las reservas se crean y se administran por transición de estado. No se elimina historial de paquetes ni reservas.

La integridad se aplica en capas:

1. Formularios: mensajes útiles para el usuario y validación de entrada.
2. Modelos: validadores, `CheckConstraint`, `UniqueConstraint` y `clean()`.
3. Servicios: reglas entre filas, publicación y cupos en `transaction.atomic`.
4. Pruebas: regresión de dominio, vistas, seguridad y concurrencia.

5.3 Ejecución y evidencia de pruebas

Comando ejecutado para esta entrega:

```
py manage.py test
```

Resultado: **16 pruebas ejecutadas, 0 errores; verificación del sistema sin incidencias.** Las pruebas cubren RUT, hash de contraseña, CSRF, límites de rol, CRUD, precio histórico, historial privado, estados de pago, idempotencia y concurrencia.

Para ejecutar la aplicación localmente:

```
python iniciar_aplicacion.py
```

El script instala la dependencia declarada si falta, aplica migraciones, guía la creación del administrador cuando corresponde y arranca el servidor.

6. Seguridad y protección de datos

Riesgo	Control implementado	Validación
Acceso no autorizado	Sesión de Django, login_required y staff_member_required.	Prueba de rutas privadas y límite cliente/personal.
Contraseñas en texto plano	set_password() y autenticación oficial de Django.	check_password() en pruebas; no se almacena contraseña ingresada.
CSRF	Middleware y tokens de formularios Django.	test_mutating_form_requires_csrf espera HTTP 403 sin token.
RUT/teléfono expuestos	No se muestran en listados; se limpian al re-renderizar un formulario inválido.	Prueba verifica ausencia de ambos valores.
Datos inválidos o duplicados	Validación de RUT, correo/RUT únicos y restricciones de BD.	Pruebas de alta y validación.
Manipulación de IDs	Las consultas de detalle se filtran por client=request.user.	Otro cliente recibe 404.
Sobreventa y doble envío	Token UUID único, transacción, validación de cupo pagado y reintentos acotados ante bloqueo SQLite.	Prueba de idempotencia y prueba concurrente.
Cambios retroactivos	Snapshots de destinos/precio y bloqueo de atributos materiales en paquetes publicados.	Prueba de precio inmutable.

Mejora recomendada para despliegue real: configurar DJANGO_SECRET_KEY y DJANGO_DEBUG=false como variables de entorno, usar HTTPS, habilitar validadores robustos de contraseña y migrar a PostgreSQL si aumentan las escrituras concurrentes. SQLite se conserva deliberadamente para el contexto académico y local.

7. Uso responsable y crítico de herramientas de IA

Se utilizó apoyo de IA como herramienta de asistencia para estructurar documentación, revisar consistencia de requisitos, proponer escenarios de prueba y detectar riesgos frecuentes de seguridad. La IA no se consideró fuente de verdad ni sustituyó la revisión del equipo.

El proceso de validación aplicado fue:

- 1. Contrastar cada propuesta con las reglas de negocio R1–R17 y con la rúbrica.
- 2. Revisar que el código respetara las APIs oficiales de Django y no expusiera información sensible.
- 3. Ejecutar pruebas automatizadas y corregir cualquier resultado no verificable.
- 4. Mantener las decisiones técnicas justificadas en el diseño: snapshots históricos, transacciones, token de idempotencia y reintentos acotados.

Ejemplos de criterio crítico: se descartó confiar solo en formularios para proteger reglas de negocio, pues pueden ser omitidos; por ello se revalidan en modelos/servicios. Asimismo, no se asumió que `select_for_update()` por sí solo resuelve SQLite: la transición de pago realiza una escritura serializadora, reintentos y una prueba de concurrencia. Estas decisiones deben poder ser explicadas por cada integrante durante la defensa.

8. Conclusión y guía para la defensa

El MVP satisface la problemática principal: centraliza el catálogo y reservas, mantiene datos históricos, protege información personal y controla el cupo bajo confirmaciones concurrentes. La documentación relaciona problema, requisito, modelo, implementación y prueba, por lo que la solución es trazable y defendible técnicamente.

Para la defensa individual se recomienda demostrar, en este orden: (1) registro de cliente y validación de RUT; (2) creación de dos destinos y un paquete; (3) publicación y evidencia de precio fijo; (4) reserva como cliente; (5) confirmación como personal y control de cupo; (6) explicación de una prueba de privacidad o concurrencia. Cada integrante debe explicar sus decisiones, límites del MVP y cómo validó el resultado, en lugar de limitarse a describir pantallas.

Anexo A. Ubicación de evidencia en el repositorio

Artefacto	Ubicación
Requisitos formales	openspec/specs/
Diseño y decisiones	openspec/changes/archive/2026-10-05-create-viajes-aventura-mvp/design.md
Matriz R1–R17	tests/TRACEABILITY.md
Pruebas	accounts/tests.py, destinations/tests.py, packages/tests.py, reservations/tests.py

Artefacto	Ubicación
Inicio y manual	iniciar_aplicacion.py, README.md
Modelos	accounts/models.py, destinations/models.py, packages/models.py, reservations/models.py
Servicios de negocio	packages/services.py, reservations/services.py